

ISTEC PORTO

Instituto Superior de Tecnologias Avançadas do Porto
CTeSP em Redes e Sistemas Informáticos · 1.º Ano · 2.º Semestre · 2025/2026
Programação Estruturada e Tipos de Dados (PETD) · Docente: Prof. Mário Amorim

FICHA DE AVALIAÇÃO — RESPOSTAS E EXPLICAÇÕES

Versão de correcção · 32 questões · Use para verificar e estudar

GRELHA RÁPIDA DE RESPOSTAS

1	2	3	4	5	6	7	8
b)	b)	c)	b)	c)	b)	a)	a)
9	10	11	12	13	14	15	16
b)	c)	a)	b)	c)	a)	b)	c)
17	18	19	20	21	22	23	24
b)	b)	b)	b)	b)	b)	b)	c)
25	26	27	28	29	30	31	32
b)	c)	c)	b)	b)	b)	b)	c)

REVISÃO DETALHADA — PERGUNTA A PERGUNTA

TIPOS DE DADOS E OPERADORES

1. Qual é o tipo de dados (type) do valor 3.14 em Python?

- a) int
- ✓ b) float
- c) double
- d) decimal

Explicação: 3.14 tem casas decimais, logo é do tipo float. Python não tem o tipo double — todos os números com vírgula flutuante são float.

2. Qual é o resultado da expressão 7 // 2 ?

- a) 3.5
- ✓ b) 3
- c) 4
- d) 1

Explicação: O operador // é a divisão inteira (descarta a parte decimal). $7 // 2 = 3$. Para o resto da divisão usa-se %.

3. Qual é o resultado da expressão 7 % 2 ?

- a) 3.5
- b) 3
- ✓ c) 1
- d) 0

Explicação: O operador % devolve o resto da divisão inteira. 7 a dividir por 2 dá quociente 3 e resto 1.

4. Qual é o resultado da expressão 2 ** 3 ?

- a) 6
- ✓ b) 8
- c) 9
- d) 23

Explicação: O operador ** é a exponenciação (potência). $2 ** 3 = 2 \times 2 \times 2 = 8$.

5. Qual destas expressões devolve True ?

- a) 5 == "5"
- b) 5 != 5
- ✓ c) 5 >= 5
- d) 5 < 5

Explicação: 5 >= 5 é verdadeiro porque 5 é igual ou maior do que 5. A primeira opção é False porque um int e uma string com o mesmo valor não são iguais em Python.

STRINGS E FORMATAÇÃO

6. Como se obtém o número de caracteres da string "Olá" ?

- a) size("Olá")
- ✓ b) len("Olá")
- c) "Olá".length
- d) count("Olá")

Explicação: A função len() devolve o comprimento de uma string, lista, tuplo ou dicionário. Em Python não existe .length como em JavaScript.

7. Qual é o resultado da expressão "abc" + "123" ?

✓ a) "abc123"

b) "abc 123"

c) Erro: tipos incompatíveis

d) 124

Explicação: O operador + entre duas strings faz concatenação (junta-as). Resulta em "abc123" sem espaço entre as duas partes.

8. Sendo nome = "Ana" e idade = 25, qual destas f-strings está correcta para mostrar "Ana tem 25 anos" ?

✓ a) f"{nome} tem {idade} anos"

b) f"nome tem idade anos"

c) "{nome} tem {idade} anos"

d) f"%nome tem %idade anos"

Explicação: Numa f-string, as variáveis vão entre chavetas {}. Sem o prefixo f, as chavetas seriam mostradas como texto literal.

9. Qual é o equivalente de f"Olá {nome}" usando o método .format() ?

a) "Olá {nome}".format()

✓ b) "Olá {}".format(nome)

c) "Olá %s".format(nome)

d) format("Olá", nome)

Explicação: Com .format(), as posições de substituição são marcadas com {} (vazias) e os valores passam-se como argumentos do método.

ENTRADAS E CONVERSÕES

10. Que tipo de dados devolve sempre a função input() ?

a) int

b) float

✓ c) str

d) depende do que o utilizador escreve

Explicação: input() devolve sempre uma string, mesmo que o utilizador escreva números. Para usar como número é preciso converter com int() ou float().

11. Qual é a forma correcta de converter a string "42" no inteiro 42 ?

✓ a) int("42")

b) integer("42")

c) (int) "42"

d) "42".toInt()

Explicação: A conversão (casting) faz-se chamando o nome do tipo como função: int("42"), float("3.14"), str(42).

12. Qual é o resultado de bool(0) ?

a) True

✓ b) False

c) 0

d) Erro

Explicação: Em Python, o número 0 é considerado falso. Outros valores que dão False: 0.0, "" (string vazia), [] (lista vazia), None.

ESTRUTURAS CONDICIONAIS

13. Qual destas estruturas if/elif/else está sintacticamente correcta ?

- a) `if x > 0 { print("positivo") }`
- b) `if (x > 0): print("positivo")`
- ✓ c) `if x > 0:
 print("positivo")`
- d) `if x > 0 then print("positivo")`

Explicação: Em Python, o `if` termina com dois pontos (`:`) e o bloco seguinte tem de estar indentado (normalmente 4 espaços). Não se usam chavetas nem palavras como `then`.

14. Sendo `x = 10`, qual é o resultado de: `x > 5 and x < 20` ?

- ✓ a) **True**
- b) False
- c) 10
- d) Erro

Explicação: Ambas as condições são verdadeiras (`10 > 5` e `10 < 20`), e o operador `and` exige que as duas sejam verdadeiras para devolver `True`.

15. Qual é o operador lógico em Python para negar uma condição ?

- a) `!`
- ✓ b) **`not`**
- c) `neg`
- d) `~`

Explicação: Em Python os operadores lógicos são palavras: `and`, `or`, `not`. O `!` é usado em outras linguagens (C, Java, JavaScript).

LISTAS, TUPLOS E DICIONÁRIOS

16. Qual destas estruturas é IMUTÁVEL (não pode ser alterada após criada) ?

- a) lista
- b) dicionário
- ✓ c) **tuplo**
- d) set

Explicação: O tuplo é imutável: depois de criado não se podem adicionar, remover nem alterar elementos. Listas e dicionários são mutáveis.

17. Como se acrescenta o valor 5 ao final da lista `numeros` ?

- a) `numeros.add(5)`
- ✓ b) **`numeros.append(5)`**
- c) `numeros.push(5)`
- d) `numeros + 5`

Explicação: O método correcto é `.append()`. O método `.add()` existe mas é para sets, e `.push()` é de outras linguagens.

18. Sendo `aluno = {"nome": "Ana", "idade": 21}`, como se obtém o valor "Ana" ?

- a) `aluno.nome`
- ✓ b) **`aluno["nome"]`**
- c) `aluno("nome")`
- d) `aluno->nome`

Explicação: Em dicionários acede-se aos valores pela chave, entre parênteses rectos: `dicionario["chave"]`. Também existe `.get("chave")` como alternativa segura.

19. Qual é o índice do primeiro elemento de uma lista em Python ?

- a) -1
- ✓ b) 0
- c) 1
- d) depende da lista

Explicação: Em Python (como na maioria das linguagens), as listas são indexadas a partir de 0. O índice -1 corresponde ao último elemento.

20. Qual é o resultado de `len([1, 2, [3, 4], 5])` ?

- a) 3
- ✓ b) 4
- c) 5
- d) Erro

Explicação: `len()` conta os elementos de topo: 1, 2, [3, 4] e 5 — quatro elementos. A sublista conta como UM elemento, mesmo tendo dois valores lá dentro.

CICLOS FOR E WHILE

21. Que valores produz `range(5)` ?

- a) 1, 2, 3, 4, 5
- ✓ b) 0, 1, 2, 3, 4
- c) 0, 1, 2, 3, 4, 5
- d) 5, 4, 3, 2, 1

Explicação: `range(n)` gera números de 0 (inclusive) até n (exclusivo). `range(5)` produz 0, 1, 2, 3, 4 — cinco números, não chega ao 5.

22. Quantas vezes é executado o bloco do ciclo:

```
for i in range(2, 8, 2):  
    print(i)
```

- a) 2
- ✓ b) 3
- c) 4
- d) 6

Explicação: `range(2, 8, 2)` gera 2, 4, 6 — três valores (começa em 2, termina antes de 8, passo de 2). Logo o `print` é executado 3 vezes.

23. Qual é a função da palavra-chave `break` dentro de um ciclo ?

- a) Salta para a próxima iteração
- ✓ b) Termina o ciclo imediatamente
- c) Pára o programa todo
- d) Reinicia o ciclo desde o princípio

Explicação: `break` sai do ciclo onde está. `continue` é que salta para a iteração seguinte. Para terminar o programa todo usa-se `exit()` ou `sys.exit()`.

24. O que acontece com o ciclo:

```
while True:  
    print("olá")
```

- a) Executa uma vez e termina
- b) Não executa nenhuma vez
- ✓ c) Entra em ciclo infinito
- d) Dá erro de sintaxe

Explicação: A condição `True` nunca se torna falsa, logo o ciclo nunca termina por si só. Para sair seria preciso um `break` dentro do bloco.

FUNÇÕES

25. Qual é a forma correcta de definir uma função em Python ?

- a) `function soma(a, b):`
- ✓ b) `def soma(a, b):`
- c) `func soma(a, b):`
- d) `def soma[a, b]:`

Explicação: Em Python usa-se a palavra-chave `def`, parênteses para os parâmetros e dois pontos no fim. `function` é de JavaScript e `func` é de Go.

26. O que devolve uma função que não tem instrução `return` ?

- a) 0
- b) False
- ✓ c) None
- d) Dá erro de execução

Explicação: Em Python, qualquer função que termine sem `return` (ou com `return` sem valor) devolve implicitamente `None` — o equivalente a "nada".

27. Considerando: `def saudar(nome, saudacao="Olá"):`

Qual destas chamadas é VÁLIDA ?

- a) `saudar()`
- b) `saudar("Bom dia")`
- ✓ c) `saudar("Ana")`
- d) `saudar(saudacao="Hi")`

Explicação: `saudar("Ana")` é válida porque `nome` é obrigatório e `saudacao` usa o valor por omissão "Olá". `saudar()` falha porque falta o `nome`.

28. O que faz o parâmetro `*args` na definição de uma função ?

- a) Indica que o argumento é obrigatório
- ✓ b) Permite receber um número variável de argumentos posicionais (recolhidos num tuplo)
- c) Define um valor por omissão
- d) Marca o argumento como ponteiro

Explicação: `*args` agrupa todos os argumentos posicionais extra num tuplo dentro da função. `**kwargs` faz o mesmo para argumentos nomeados, agrupando-os num dicionário.

MÓDULOS, ERROS E FICHEIROS

29. Qual é a diferença entre `import math` e `from math import sqrt` ?

- a) Não há diferença
- ✓ b) `import math` importa todo o módulo (uso: `math.sqrt`); `from math import sqrt` importa só a função (uso: `sqrt` directamente)
- c) `from math import sqrt` é mais rápido a executar
- d) `import math` só funciona em Python 2

Explicação: Com `import math` é preciso prefixar com `math.` para chamar funções; com `from math import sqrt` a função fica disponível directamente como `sqrt()`, sem prefixo.

30. Qual destes blocos trata correctamente um erro de conversão de string para int ?

a) `if int(input()): print("ok")`

✓ b) `try:
 n = int(input())
except ValueError:
 print("valor inválido")`

c) `catch (ValueError):
 print("erro")`

d) `on error: print("erro")`

Explicação: Em Python o tratamento de erros faz-se com `try/except`. `catch` é de Java/C++/JavaScript. `ValueError` é a excepção lançada quando `int()` recebe uma string que não representa um número.

31. Qual é a forma RECOMENDADA de abrir um ficheiro de texto para leitura em Python ?

a) `f = open("dados.txt", "r")
... f.close()` manual

✓ b) `with open("dados.txt", "r") as f:
 ... (fecha automaticamente)`

c) `f = file("dados.txt")`

d) `f = read("dados.txt")`

Explicação: A construção `with open() ... as f:` garante que o ficheiro é fechado automaticamente, mesmo se ocorrer um erro. É a forma idiomática em Python.

32. Qual é o modo de abertura de ficheiro que apaga o conteúdo existente e escreve do princípio ?

a) "r" (read)

b) "a" (append)

✓ c) "w" (write)

d) "x" (exclusive)

Explicação: "w" (write) trunca o ficheiro — apaga tudo o que lá estava. "a" (append) acrescenta no fim sem apagar. "x" cria novo, mas dá erro se já existir.